

بدل Tom Select الخاصة بالمحطات باستخدام مكتبة اسمها Searchable Dropdowns لل Feature أنا عملت jQuery ، لأنها خفيفة وسريعة ومش محتاجة Select2 المكتبات الثقيلة زي الفكرة إن المستخدم يقدر يكتب اسم المحطة ويلاقها بسرعة مع دعم كامل للكيورد

وخليت المحطات تترتب أبجدي باستخدام Controllers في الـ LINQ Query عدلت الـ Backend ومن ناحية الـ

OrderBy(s => s.Name)

أحسن UX وده خلّى البحث والـ

فضل ASP.NET Core بتاع POST Logic العادي، يعني الفورم والـ <select> شغال فوق الـ Tom Select كمان الـ Business Logic أو الـ Routing شغال زي ما هو من غير ما أغير أي حاجة في الـ

---

بإستخدام GPS لل Feature بعد كده عملت:

- OpenStreetMap
- Leaflet.js

ومجاني Open Source عشان يكون الحل Google Maps APIs بدل

وضفت ViewModel أول حاجة عدلت الـ

- Latitude
- Longitude

Frontend. عشان أبعت إحداثيات المحطات للـ

في Circular References آمن عشان أتجنب مشاكل الـ JSON Array في شكل JavaScript بعدها بعث البيانات للـ ASP.NET.

---

"Use My Location": لما المستخدم يضغط على زر ار

- navigator.geolocation.getCurrentPosition() يستخدم
- بتوع المستخدم Longitude والـ Latitude وباخد الـ

Haversine Formula بعدها بحسب أقرب محطة باستخدام الـ

بتطلع Pythagorean theorem لأن الأرض كروية، فالحسابات العادية زي الـ Haversine استخدمت الـ inaccurate في المسافات الحقيقية

بيلف على كل المحطات الـ Algorithm

- يحسب المسافة

- كأقرب محطة Distance ويختار أقل
- 

بعد ما أحدد أقرب محطة

- Tom Select بتاع API باستخدام Departure Dropdown تلقائي للـ Update بعمل
- لمكان المستخدم Marker ويرسم
- للمحطة Marker و
- Polyline ويوصل بينهم بخط باستخدام

وفي الآخر بستخدم

map.fitBounds()

.عشان أخلي الاتنين ظاهرين في الشاشة بشكل تلقائي

---

Error Handling: كمان ضفت

- GPS Permission لو المستخدم رفض الـ
- أخذ وقت طويل GPS أو الـ

.مناسبة Error Message بيظهرله

في Features وفي الآخر كررت نفس الـ

- Routes/Index.cshtml
- Home/Index.cshtml و

.يبقى موحد في الموقع كله UX عشان الـ

:والأهم إنني ما لمستش

- Database Structure
- Routing Logic
- Pricing Logic

.بالكامل Stable فالنظام الأساسي فضل